Principal Component Analysis

PCA in an unsupervisioned tecnique which aims to reduce the number of features of the data keeping as much as possible of the total variance.
It wants to find a subspace $P \in \mathbb{R}^{n \times m}$ where the columns of P are orthonormal so that $PP^\top = I$ and form a basis of $\mathbb{R}^n$.
The projection of a point on P is defined as $y = P^\top x$ and its reconstruction in the original space is:

$$x = Py \quad \Rightarrow \quad \hat{x} = PP^\top x$$

We want P so that it minimizes the reconstruction error:

$$P = \arg\min_P \frac{1}{N} \sum_{i=1}^{N} \left\| x_i - PP^\top x_i \right\|^2$$

It turns out the best P matrix is formed by the m eigenvectors associated to the biggest eigenvalues of the covariance matrix of the data.

$$C = \frac{1}{K} \sum_i x_i x_i^\top = U \Sigma U^\top$$

$$\Sigma = \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_n \end{bmatrix} \quad \text{with} \quad \sigma_1 > \sigma_2 > \cdots > \sigma_n$$

This procedure assumes data are centered, if it's not the case the first direction of P will be a vector pointing toward the mean of the data cluster.
To address this problem, we subtract from each point the data mean:

$$C = \frac{1}{N} \sum_i (x_i - \mu)(x_i - \mu)^\top \quad \text{where} \quad \mu = \frac{1}{N} \sum_i x_i$$

PCA can be interpreted as a linear mapping to a space which preserves only the directions with highest variance.
If the dimension of the destination space is the same as the original one, it actually performs a rotation of the data.
m is a hyperparameter and it's possible to choose a value by cross-validation or choose it so that its minimum value maintains the total variance above a threshold.

$$m = \min_m \left(\frac{\sum_{i=1}^{m} \sigma_i}{\sum_{i=1}^{n} \sigma_i} \right) \geq t$$

As it's been already said, PCA doesn't use class labels, so it can't be used to address classification problems. Moreover it's susceptible to scaling of the features.

Linear Discriminant Analysis

LDA is a supervised learning technique that seeks directions in the feature space that best discriminate between classes. As a byproduct, it can also be used for dimensionality reduction, projecting data into a lower-dimensional space with high class separability.
The idea is to find the directions which better allow to separate the classes. A first approach could be to project data onto the direction joining the means of the two classes, but if points of the classes are scattered along that direction, it would not behave differently from PCA, not fully using information about class labels.
Then the final goal becomes to maximize the ratio of the covariance between the classes over the covariance within the classes:

$$\mathcal{L}(w) = \max_w \frac{w^\top S_B w}{w^\top S_W w}$$

where:

$$S_B = \frac{1}{N}\sum_{i=1}^{N_c} n_i(\mu_i - \mu)(\mu_i - \mu)^\top$$

$$S_W = \frac{1}{N}\sum_{i=1}^{N_c}\sum_{j=1}^{n_i}(x_{ij} - \mu_i)(x_{ij} - \mu_i)^\top$$

Computing the gradient of $\mathcal{L}(w)$ and setting it equal to $\varnothing$ we get the standard equation for eigenvalues:

$$\nabla_w \left(\frac{w^\top S_B w}{w^\top S_W w}\right) = \varnothing$$

$$S_B w = \lambda S_W w \quad \text{where} \quad \lambda = \frac{w^\top S_B w}{w^\top S_W w}$$

$$\Rightarrow \lambda w = S_W^{-1} S_B w$$

It follows that the optimal vector w is eigenvector of $S_W^{-1}S_B$ with λ an eigenvalue.

To derive a classifier from LDA, it assigns a point x_t to the class with the closest mean.
Imagining that μ_1 is farther than μ_0:

$$C(x_t) = \begin{cases} C_0 & \text{if } \|w^\top x_t - \mu_0\|^2 < \|w^\top x_t - \mu_1\|^2 \\ C_1 & \text{else} \end{cases}$$

$$\bar{\mu}_0 = w^\top \mu_0 \quad \text{and} \quad \bar{\mu}_1 = w^\top \mu_1 \quad \text{(projection of the means on } w\text{)}$$

$$(w^\top x_t)^2 + \bar{\mu}_0^2 - 2w^\top x_t \bar{\mu}_0 < (w^\top x_t)^2 + \bar{\mu}_1^2 - 2w^\top x_t \bar{\mu}_1$$

$$\Rightarrow (\bar{\mu}_0^2 - \bar{\mu}_1^2) - 2w^\top x_t(\bar{\mu}_0 - \bar{\mu}_1) < 0$$
$$\Rightarrow w^\top x_t < t \quad \text{where} \quad t = \frac{\bar{\mu}_0 + \bar{\mu}_1}{2}$$

This corresponds to a linear classifier with decision surfaces orthogonal to w.

Fow how S_B it's defined the number of distinct eigenvalues different from 0 is $C - 1$ where C is the number of classes.

In many applications, it is useful to pre-process data with PCA to remove features with low variance and then classify with LDA.

If data have a non-linear pattern, it's possible to transform the coordinate system, for example from Cartesian to Polar.

Connection with Tied Gaussian Classifier

LDA can be derived as the Bayes-optimal classifier under the assumption that the class-conditional distributions are Gaussian with the same covariance matrix across all classes (i.e., a Tied Gaussian Model).
In this case, the log-likelihood ratio between two classes reduces to a linear function:

$$\log \frac{p(x \mid c_1)}{p(x \mid c_0)} = x^\top \Sigma^{-1}(\mu_1 - \mu_0) - \frac{1}{2}(\mu_1^\top \Sigma^{-1}\mu_1 - \mu_0^\top \Sigma^{-1}\mu_0)$$

We can rewrite this expression in the form of a linear function:

$$\log \frac{p(x \mid c_1)}{p(x \mid c_0)} = x^\top A + b$$

where:

$$A = \Sigma^{-1}(\mu_1 - \mu_0) \quad \text{and} \quad b = -\frac{1}{2}\left(\mu_1^\top \Sigma^{-1}\mu_1 - \mu_0^\top \Sigma^{-1}\mu_0\right)$$

This reveals that the decision surface of the classifier is linear in x, as in LDA.
This connection also shows that LDA can be interpreted as both a discriminative dimensionality reduction method and a generative linear classifier derived from the assumption of Gaussian class distributions with tied covariance.
Consequently, LDA performs well when the classes are approximately Gaussian and share a similar covariance structure.

Multivariate Gaussian Classifiers

MVG assumes feature vectors are described by multivariate normal distributions, conditioned on the class label:

$$(x|c) \sim \mathcal{N}(x|\mu_c, \Sigma_c)$$

The goal of this model is to estimate the parameters:

$$\Theta = [(\mu_1, \ldots, \mu_{n_c}), (\Sigma_1, \ldots, \Sigma_{n_c})]$$

which better approximate the class distributions which generated the data.
To do it, we define the likelihood function as a measure of how likely to observe x belonging to c_i, with parameters Θ:

$$\mathcal{L}(\Theta) = \prod_i^n f(x_i, c_i|\Theta)$$

$$= \prod_i^n f(x_i|c_i, \Theta) \cdot P(c_i)$$

$f(x_i|c_i; \Theta)$ is the class conditional density and $P(c_i)$ is the prior, an application dependent parameter.

To avoid numerical issues due to floating point approximation (the function multiplies terms that are probabilities or density of probabilities, so values constrained between 0 and 1), we take the logarithm of likelihood $\ell(\Theta)$:

$$\ell(\Theta) = \sum_i \log \mathcal{N}(x_i|\mu_{c_i}, \Sigma_{c_i}) + \sum_i \log P(c_i)$$

$$= \sum_{k=1}^{C} \sum_{i \in c_k} \log \mathcal{N}(x_i|\mu_k, \Sigma_k) + \xi$$

$$= \sum_{k=1}^{C} \ell_k(\Theta) + \xi$$

It turns out we can maximize $\ell(\Theta)$ by maximizing class-independent likelihood, while $P(c_i)$ does not depend on Θ.

Taking the gradient of $\ell_k(\Theta)$ and setting it equal to 0, we get the Maximum Likelihood solutions for μ_c and Σ_c:

$$\mu_c^* = \frac{1}{n_c} \sum_{i \in c} x_i$$

$$\Sigma_c^* = \frac{1}{n_c} \sum_{i \in c} (x_i - \mu_c^*)(x_i - \mu_c^*)^\top$$

Once the parameters have been estimated, we can assign a test point to the class which has the highest posterior probability. That one is the ratio of joint density over the marginal density, which is the sum of the joint density over all the classes.

$$c^* = \arg\max_c P(c \mid x)$$

For a binary classification problem, we can compute the ratio of the two posteriors:

$$r(x) = \frac{P(c_1|x)}{P(c_0|x)} > 1$$

$$\log r(x) = \log \frac{P(x|c_1)}{P(x|c_0)} + \log \frac{P(c_1)}{P(c_0)} > 0$$

The first term is the log-likelihood ratio, $\ell\ell(x)$, while the second one is the priors' log-odds.

Since there are two classes:

$$P(c_1) = \pi \quad \Rightarrow \quad P(c_0) = 1 - \pi$$

$$\ell\ell(x) > -\log \frac{\pi}{1-\pi}$$

The priors act as a threshold which moves as the priors change, with the probabilistic interpretation that stronger the priors for one class, greater the evidence for the other to let the model "change idea".

$$\ell\ell(x) = \log \left(\frac{\mathcal{N}(x|\mu_1, \Sigma_1)}{\mathcal{N}(x|\mu_0, \Sigma_0)} \right)$$

$$= \frac{d}{2}\log 2\pi + \frac{1}{2}\left[\log|\Sigma_0| - (x-\mu_0)^\top \Sigma_0^{-1}(x-\mu_0)\right] - \frac{d}{2}\log 2\pi - \frac{1}{2}\left[\log|\Sigma_1| - (x-\mu_1)^\top \Sigma_1^{-1}(x-\mu_1)\right]$$

$$\Rightarrow \ell\ell(x) = x^\top A x + b^\top x + c$$

$$A = -\frac{1}{2}(\Sigma_1^{-1} - \Sigma_0^{-1}) \qquad b = \Sigma_1^{-1}\mu_1 - \Sigma_0^{-1}\mu_0 \qquad c = -\frac{1}{2}\left(\mu_1^\top \Sigma_1^{-1}\mu_1 - \mu_0^\top \Sigma_0^{-1}\mu_0\right)$$

The decision surface is quadratic in x.

Naive Bayes MVG

Assume no correlation between the features for each class. This forces the covariance matrix to be diagonal:

$$\Sigma_c = \begin{bmatrix} \sigma_{c,1} & 0 & \cdots & 0 \\ 0 & \sigma_{c,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_{c,m} \end{bmatrix}$$

The reduction of number of parameters strongly simplifies the computation. In fact, the density of $(x|c)$ can be factorized over its components:

$$f(x|c) = \mathcal{N}(x|\mu_c, \Sigma_c) = \prod_j \mathcal{N}(x_{[j]}|\mu_{c[j]}, \Sigma_{c[j]})$$

This model could perform poorly if there is correlation between the features, as often appears in real-world scenarios.

Tied MVG

This model instead assumes all classes share the same covariance matrix, meaning the features can be correlated, but all the classes have similar distribution. In some cases, this assumption can work pretty well:

$$f(x|c) = \mathcal{N}(x|\mu_c, \bar{\Sigma})$$

In this case, the decision surface is linear since the quadratic term simplifies:

$$A = -\frac{1}{2}(\Sigma_1^{-1} - \Sigma_0^{-1}) = \frac{1}{2}(\Sigma^{-1} - \Sigma^{-1}) = 0$$

$$\Rightarrow \ell\ell(x) = x^T b + c$$

Model Evaluation

To evaluate the performance of a classification model, we typically define two basic metrics:

- **Accuracy:** the proportion of correctly classified samples.

$$\text{accuracy} = \frac{\#\text{correct predictions}}{\#\text{total predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Error rate:** the proportion of misclassified samples.

$$\text{error rate} = \frac{\#\text{wrong predictions}}{\#\text{total predictions}} = 1 - \text{accuracy}$$

Where:

- TP: True Positives
- TN: True Negatives
- FP: False Positives
- FN: False Negatives

These quantities are derived from the confusion matrix of a binary classifier.

To remove dependency from the class priors, we define error rates conditioned on the class:

$$P_{fn} = \frac{FN}{FN + TP}, \quad P_{fp} = \frac{FP}{FP + TN}, \quad P_{tp} = \frac{TP}{TP + FN}, \quad P_{tn} = \frac{TN}{TN + FP}$$

The global error rate can be written as a weighted average of class-specific error rates:

$$\text{error rate} = \frac{FN + FP}{TP + TN + FP + FN}$$

By adding and subtracting quantities appropriately, we can express this as:

$$\text{error rate} = \frac{FN}{FN + TP + TN + FP} + \frac{FP}{FN + TP + TN + FP}$$

Then we reorganize as a weighted sum:

$$\text{error rate} = P_{fn} \cdot \pi^{\text{emp}} + P_{fp} \cdot (1 - \pi^{\text{emp}})$$

Where:

$$\pi^{\text{emp}} = \frac{FN + TP}{FN + TP + TN + FP}$$

is the empirical prior of the positive class, i.e., the fraction of positive samples in the dataset.
We obtain a weighted sum of class-wise error rates, where the weights are the empirical priors. This allows us to fairly evaluate classifiers on imbalanced datasets, decouple performance metrics from class distributions and better understand the sources of error per class.

Bayes Decision and Bayes Risk

The goal of a classifier R, beyond just assigning labels, is to decide whether or not to take an action $a \in \mathcal{A}$, from a set of possible actions, where each action incurs a cost depending on the true class C of the input x.
This cost is denoted by:

$$C(a(x|R) \mid c)$$

which represents the cost of taking action a (suggested by recognizer R) when the actual class is c.

The Bayes Risk is the expected cost, i.e., the theoretical mean of the risk:

$$\mathcal{B} = \sum_{c=1}^{K} \pi_c \int f(x \mid c) C(a(x|R) \mid c)\, dx$$

where: π_c is the prior probability of class c, $f(x \mid c)$ is the class-conditional density and $C(a(x|R) \mid c)$ is the cost of the decision given the true class.

In real-world scenarios, we don't know the true distributions $f(x \mid c)$, but we have access to labeled samples from which we can approximate the risk empirically.

Let N_c be the number of samples from class c. Then:

$$\mathcal{B}_{\text{emp}} = \sum_{c=1}^{K} \pi_c^{\text{emp}} \cdot \frac{1}{N_c} \sum_{i \in c} C(a(x_i \mid R) \mid c)$$

where $\pi_c^{\text{emp}} = \frac{N_c}{N}$ is the empirical class prior.

For the binary case the empirical Bayes risk simplifies to:

$$\mathcal{B}_{\text{emp}} = \pi C_{fn} P_{fn} + (1 - \pi) C_{fp} P_{fp}$$

This formula makes it explicit how classification errors and cost asymmetries influence the total expected loss.

Cost Detection Function (DCF)

The DCF (Cost Detection Function) represents the mean cost associated with classification decisions.
It is also referred to as the unnormalized DCF, and it can be normalized by comparing it with the best dummy system.

A dummy system always predicts one class based on the priors π. Thus:

$$\begin{cases} P_{fn} = 1, \quad P_{fp} = 0 & \Rightarrow \text{DCF} = (1 - \pi) \cdot C_{fp} \quad \text{(always predicts } H_1\text{)} \\ P_{fn} = 0, \quad P_{fp} = 1 & \Rightarrow \text{DCF} = \pi \cdot C_{fn} \quad \text{(always predicts } H_0\text{)} \end{cases}$$

The normalized DCF is defined as the ratio between the actual DCF and the best dummy DCF:

$$\text{DCF}_{\text{norm}}(\pi, C_{fn}, C_{fp}) = \frac{\text{DCF}_{\text{actual}}(\pi, C_{fn}, C_{fp})}{\min(\pi \cdot C_{fn}, (1 - \pi) \cdot C_{fp})}$$

This normalized version is invariant to scaling and gives a better sense of model quality compared to a trivial system.

Threshold Based on Cost and Priors

The posterior ratio $r(x)$ can be expressed in terms of costs, priors, and likelihoods:

$$\log r(x) = \log\left(\frac{C_{fn}}{C_{fp}} \cdot \frac{P(H_1 \mid x)}{P(H_0 \mid x)}\right) = \log\left(\frac{f(x \mid H_1)}{f(x \mid H_0)}\right) + \log\left(\frac{C_{fn} \cdot \pi}{(1-\pi) \cdot C_{fp}}\right)$$

From this, we derive the optimal log-likelihood ratio threshold:

$$\ell\ell r(x) \geq t \quad \text{where} \quad t = \log\left(\frac{C_{fn} \cdot \pi}{(1-\pi) \cdot C_{fp}}\right)$$

In practice, systems often produce log-likelihood ratios $\ell\ell r(x)$ that are not well-calibrated, due to lack of probabilistic interpretability, mismatch between training and testing distributions, wrong assumptions in the model.

We define:

- DCF_{min}: the minimum achievable cost using threshold t computed from prior and cost, reflects the discriminative potential of the model.
- DCF_{act}: the actual cost obtained using the threshold t derived from the effective priors.

The difference $\text{DCF}_{\text{act}} - \text{DCF}_{\text{min}}$ represents the loss due to miscalibration.

Logistic Regression

Logistic Regression is a widely used model for binary classification. Unlike generative models such as MVG or LDA, which estimate the class-conditional distributions and then apply Bayes' theorem to compute posteriors, logistic regression directly models the posterior probability $P(H_1 \mid x)$ as a function of the input. It is a discriminative model that provides probabilistic outputs and avoids the need to explicitly estimate the distribution of the data. One of the key advantages of this approach is that the decision surface (or threshold) is defined directly by the model, without passing through the intermediate step of likelihood estimation. Furthermore, class priors are implicitly absorbed into the learned parameters ω and b, simplifying the decision function and making it independent of explicit prior estimation.

Logistic Regression can be motivated starting from the Tied MVG model, where we assume equal covariance matrices for all classes. In this case, the log-likelihood ratio becomes a linear function:

$$\ell\ell r(x) = \log\left(\frac{P(H_1 \mid x)}{P(H_0 \mid x)}\right) = \omega^T x + b$$

Using the fact that $P(H_1 \mid x) + P(H_0 \mid x) = 1$, we derive:

$$\frac{P(H_1 \mid x)}{1 - P(H_1 \mid x)} = e^{\omega^T x + b}$$

Solving for $P(H_1 \mid x)$:

$$P(H_1 \mid x) = \frac{e^{\omega^T x + b}}{1 + e^{\omega^T x + b}} = \frac{1}{1 + e^{-\omega^T x - b}} = \sigma(\omega^T x + b)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.

The sigmoid function squashes any real-valued input to a value between 0 and 1, making it suitable for modeling probabilities. Logistic regression thus defines a probabilistic decision boundary based on a linear combination of the input features. It also enables a smooth transition between class decisions as a function of input confidence, unlike hard thresholding methods.

We train the model by maximizing the likelihood of the observed labels given the inputs. Assuming independent data points (x_i, c_i):

$$\mathcal{L}(\theta) = \prod_i P(c_i \mid x_i; \theta) f(x_i)$$

Taking the log-likelihood and dropping the marginal term $f(x_i)$ (which does not depend on θ):

$$\ell(\theta) = \sum_i \log P(c_i \mid x_i)$$

Let $y_i = \sigma(\omega^T x_i + b)$, and $c_i \in \{0, 1\}$ be the true class label following a Bernoulli distribution. Then:

$$\mathcal{L}(\omega, b) = \prod_i y_i^{c_i} (1 - y_i)^{1 - c_i}$$

$$\ell(\omega, b) = \sum_i \left[c_i \log y_i + (1 - c_i) \log(1 - y_i)\right]$$

The function to be minimized (negative log-likelihood) is:

$$\mathcal{J}(\omega, b) = -\ell(\omega, b) = \sum_i \log\left(1 + e^{-z_i(\omega^T x_i + b)}\right)$$

where $z_i = 2c_i - 1 \in \{-1, 1\}$.

This formulation allows us to interpret the logistic loss as an empirical risk. Even correct predictions are penalized if made with low confidence:

- $z_i s_i > 0$: correct prediction, small risk.
- $z_i s_i < 0$: wrong prediction, large risk.

where $s_i = \omega^T x_i + b$. The loss function is:

$$\mathcal{H}(c_i, y_i) = \log(1 + e^{-z_i s_i})$$

The graph of this loss reveals that, unlike the 0-1 loss which is discontinuous, the logistic loss is smooth and convex. It penalizes highly confident wrong predictions more severely than low-confidence ones.

When the training data is linearly separable, logistic regression has no finite optimum: if (ω, b) separates the data, so does $(2\omega, 2b)$, and so on. The model keeps increasing $\|\omega\|$ to minimize the loss:

$$\|\omega\| \to \infty, \quad \mathcal{J}(\omega, b) \to 0$$

This leads to overfitting and poor generalization, especially with small datasets or when decision boundaries are sharp and close to data points.

To counteract this, a regularization term is introduced:

$$\widetilde{\mathcal{R}}(\omega, b) = \frac{\lambda}{2}\|\omega\|^2 + \sum_i \log\left(1 + e^{-\tilde{c}_i(\omega^T x_i + b)}\right)$$

Where λ is a hyperparameter that controls model complexity:

- If λ is too large: underfitting, poor separation.
- If λ is too small: overfitting, poor generalization.

This regularized version of logistic regression balances data fitting and model simplicity. It generalizes better and avoids degeneracies in separable data. The final model not only provides probabilistic predictions, but also embeds a notion of margin and confidence, making it a foundational tool in modern machine learning pipelines.

Multiclass Logistic Regression

We now extend logistic regression to the multiclass setting. Given K classes labeled $1, 2, \ldots, K$, the objective is to estimate the posterior class probabilities $P(C = k \mid x)$ directly, without assuming an underlying generative model for the features.

From a generative viewpoint, such as the tied covariance Gaussian classifier, we know that the log-likelihood can be expressed in the form:

$$\log f_{X|C}(x \mid c) = x^\top w_c + b_c + k(x),$$

where $w_c = \Lambda\mu_c$ and $b_c = -\frac{1}{2}\mu_c^\top \Lambda \mu_c$, with Λ being the shared precision matrix and $k(x)$ independent of the class label. This motivates the discriminative formulation of multiclass logistic regression:

$$\log P(C = c \mid x) = w_c^\top x + b_c - \log Z(x),$$

where $Z(x) = \sum_{k=1}^{K} e^{w_k^\top x + b_k}$ ensures that the posterior probabilities sum to 1. This gives rise to the softmax function:

$$P(C = c \mid x) = \frac{e^{w_c^\top x + b_c}}{\sum_{j=1}^{K} e^{w_j^\top x + b_j}}.$$

Letting $S(x) = [s_1(x), \ldots, s_K(x)]$ be the score vector where $s_k(x) = w_k^\top x + b_k$, the model's output is:

$$P(C = c \mid x) = \text{softmax}(S(x))_c.$$

We define the log-likelihood over a dataset $\{(x_i, c_i)\}_{i=1}^n$ using one-hot encoded vectors z_i for each label c_i:

$$\ell(W, b) = \sum_{i=1}^{n} \sum_{k=1}^{K} z_{ik} \log y_{ik},$$

where $y_{ik} = \text{softmax}(Wx_i + b)_k$. The associated cross-entropy loss per sample is:

$$H(z_i, y_i) = -\sum_{k=1}^{K} z_{ik} \log y_{ik},$$

and the overall training objective becomes the minimization of the average cross-entropy:

$$\min_{W,b} \frac{1}{n} \sum_{i=1}^{n} H(z_i, y_i).$$

Since the model is overparameterized, it is common to constrain the weights (e.g., by fixing one class as reference) to remove redundancy. Regularization is added as:

$$R(W, b) = \Omega(W) + \frac{1}{n} J(W, b),$$

where a typical choice is $\Omega(W) = \frac{\lambda}{2}\|W\|^2$.

Unlike generative models, logistic regression does not attempt to model the data distribution $f_X(x)$ or class-conditional likelihoods $f_{X|C}(x|c)$. This makes it discriminative by nature: it focuses only on estimating decision boundaries. The class priors are implicitly absorbed into the weights and biases during training, and decision surfaces are linear in x.

Quadratic Surfaces and Expanded Feature Spaces

If the class-conditional distributions differ in covariance matrices (e.g., Gaussian classifiers without tied covariance), the posterior log-ratio becomes quadratic:

$$\log \frac{P(C=1 \mid x)}{P(C=0 \mid x)} = x^\top A x + b^\top x + c.$$

To model this discriminatively, we can map the input x to a higher-dimensional feature space using a transformation $\varphi(x)$:

$$\varphi(x) = \begin{bmatrix} \mathrm{vec}(xx^\top) \\ x \end{bmatrix}, \quad w = \begin{bmatrix} \mathrm{vec}(A) \\ b \end{bmatrix},$$

leading to a linear model in the transformed space:

$$s(x) = w^\top \varphi(x) + c.$$

Training proceeds as in standard logistic regression, but now the classifier operates in an expanded feature space where quadratic decision boundaries in x become linear in $\varphi(x)$.

This method is powerful but computationally demanding, as the dimension of $\varphi(x)$ grows quadratically (or worse) with the input size. It provides a flexible way to approximate more complex decision boundaries without generative assumptions.

Experiments on datasets like MNIST show that while standard logistic regression often outperforms simple Gaussian models (due to weak Gaussian assumptions), the use of quadratic expansions or PCA-preprocessing with Gaussian models can provide superior performance when nonlinear separations are more appropriate.

Support Vector Machines

Support Vector Machines (SVMs) provide an alternative approach to binary classification that offers a geometrical interpretation of regularization. While Logistic Regression maximizes the log-probability of class assignments by minimizing the logistic loss, SVMs select the hyperplane that separates classes with the largest margin.

Like Logistic Regression, SVMs can be formulated as a regularized risk minimization problem:

$$\min_{w,b} \frac{\lambda}{2}\|w\|^2 + \frac{1}{n}\sum_{i=1}^{n} \ell(z_i(w^T x_i + b))$$

where ℓ is the loss function and $z_i \in \{-1, +1\}$ are the class labels.

However, unlike Logistic Regression, SVM outputs cannot be directly interpreted as class posteriors. The key advantage of SVMs lies in their natural extension to non-linear separation through the kernel trick, without requiring explicit feature expansion.

Hard Margin SVM

For linearly separable classes, the hard margin SVM selects the hyperplane that maximizes the margin, defined as the distance of the closest point to the separation hyperplane.

Given the separation function $f(x) = w^T x + b$, the distance of point x_i from the hyperplane is:

$$d(x_i) = \frac{|z_i(w^T x_i + b)|}{\|w\|}$$

The maximum margin hyperplane is found by solving:

$$w^*, b^* = \arg\max_{w,b} \min_{i \in \{1,\ldots,n\}} \frac{z_i(w^T x_i + b)}{\|w\|}$$

Due to the scale invariance of the objective function, we can normalize the parameters such that $\min_i z_i(w^T x_i + b) = 1$. This transforms the problem into the equivalent constrained optimization:

Primal formulation:

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2$$

$$\text{s.t.} \quad z_i(w^T x_i + b) \geq 1, \quad i = 1, \ldots, n$$

This is a convex quadratic programming problem. Using Lagrange multipliers $\alpha_i \geq 0$, the Lagrangian is:

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i [z_i(w^T x_i + b) - 1]$$

Setting the derivatives with respect to w and b to zero yields:

$$w = \sum_{i=1}^{n} \alpha_i z_i x_i$$

$$0 = \sum_{i=1}^{n} \alpha_i z_i$$

Substituting these constraints into the Lagrangian gives the **dual formulation**:

$$\max_{\alpha} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j z_i z_j x_i^T x_j$$

$$\text{s.t.} \quad \alpha_i \geq 0, \quad i = 1, \ldots, n$$

$$\sum_{i=1}^{n} \alpha_i z_i = 0$$

The optimal solution satisfies the Karush-Kuhn-Tucker (KKT) conditions. Points with $\alpha_i \neq 0$ lie exactly on the margin and are called **support vectors**. For a test point x_t, the classification score is:

$$s(x_t) = \sum_{i=1}^{n} \alpha_i z_i x_i^T x_t + b$$

Notice that both training and prediction depend only on dot products $x_i^T x_j$.

Soft Margin SVM

For non-linearly separable classes, we introduce slack variables $\xi_i \geq 0$ to allow violations of the margin constraint. The constraint becomes:

$$z_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

The **soft margin SVM** minimizes:

$$\min_{w,b,\xi} \quad \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{n} \xi_i$$

$$\text{s.t.} \quad z_i(w^T x_i + b) \geq 1 - \xi_i, \quad \forall i$$

$$\xi_i \geq 0, \quad \forall i$$

where C controls the trade-off between margin maximization and training error minimization. The dual formulation remains similar to the hard margin case:

$$\max_{\alpha} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j z_i z_j x_i^T x_j$$

$$\text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad i = 1, \ldots, n$$

$$\sum_{i=1}^{n} \alpha_i z_i = 0$$

By eliminating the slack variables, the primal problem can be rewritten as:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \max[0, 1 - z_i(w^T x_i + b)]$$

The function $\max(0, 1-s)$ is called the **hinge loss**. This formulation is remarkably similar to regularized Logistic Regression:

$$\text{SVM:} \quad \min_{w,b} \frac{\lambda}{2}\|w\|^2 + \frac{1}{n}\sum_{i=1}^{n} \max[0, 1 - z_i(w^T x_i + b)]$$

$$\text{LR:} \quad \min_{w,b} \frac{\lambda}{2}\|w\|^2 + \frac{1}{n}\sum_{i=1}^{n} \log[1 + e^{-z_i(w^T x_i + b)}]$$

The key difference lies in the loss function: SVMs use the hinge loss while Logistic Regression uses the logistic loss.

Kernel Methods

The power of SVMs lies in their ability to achieve non-linear separation without explicit feature expansion. Since both training and prediction depend only on dot products, we can define a **kernel function**:

$$k(x_1, x_2) = \Phi(x_1)^T \Phi(x_2)$$

where Φ is an implicit mapping to a higher-dimensional feature space.

The dual formulation becomes:

$$H_{ij} = z_i z_j k(x_i, x_j)$$

and the prediction function is:

$$s(x_t) = \sum_{i:\alpha_i > 0} \alpha_i z_i k(x_i, x_t) + b$$

Common kernel functions include:

- **Polynomial kernel:** $k(x_1, x_2) = (x_1^T x_2 + 1)^d$
- **RBF kernel:** $k(x_1, x_2) = e^{-\gamma\|x_1 - x_2\|^2}$

This **kernel trick** enables training in high-dimensional spaces with computational complexity depending only on the number of training samples, not the feature space dimension.

While Logistic Regression can also incorporate kernels, SVMs provide a more natural framework for non-linear classification through the dual formulation, making them particularly effective for complex decision boundaries.

Gaussian Mixture Models

One significant limitation of generative models like Multivariate Gaussian (MVG) is their assumption of a specific distribution that may not accurately represent the data. Gaussian Mixture Models (GMMs) provide an alternative approach to model generic distributions by approximating any sufficiently regular distribution to a desired degree of accuracy.

GMMs can be used for various tasks beyond density estimation, including clustering as an alternative to K-means. A Gaussian Mixture Model represents a density as a weighted combination of Gaussian components:

$$X \sim GMM(M, S, w)$$

$$f_X(x) = \sum_{c=1}^{K} w_c \mathcal{N}(x; \mu_c, \Sigma_c)$$

where $M = [\mu_1, \ldots, \mu_K]$ are the component means, $S = [\Sigma_1, \ldots, \Sigma_K]$ are the component covariances, and $w = [w_1, \ldots, w_K]$ are the mixture weights with $\sum_{c=1}^{K} w_c = 1$.

We have already encountered GMMs in the context of Gaussian classification, where the marginal distribution is computed as:

$$f_X(x) = \sum_{c=1}^{K} \pi_c \mathcal{N}(x|\mu_c, \Sigma_c)$$

Hard Cluster Assignments

Given a dataset $D = [x_1, \ldots, x_n]$ generated independently by a GMM, we can estimate the model parameters $\theta = [M, S, w]$ using Maximum Likelihood estimation. However, direct ML estimation for GMMs is ill-posed due to potential degenerate solutions with unbounded likelihood.

The likelihood function is:

$$L(\theta) = \prod_{i=1}^{n} f_{X_i}(x_i) = \prod_{i=1}^{n} \sum_{c=1}^{K} w_c \mathcal{N}(x_i|\mu_c, \Sigma_c)$$

with corresponding log-likelihood:

$$\ell(\theta) = \sum_{i=1}^{n} \log \left(\sum_{c=1}^{K} w_c \mathcal{N}(x_i|\mu_c, \Sigma_c) \right)$$

A GMM can be interpreted as the marginal of a joint distribution of data points and corresponding clusters:

$$f_{X_i}(x_i) = \sum_{c=1}^{K} f_{X_i|C_i}(x_i|c) P(C_i = c) = \sum_{c=1}^{K} w_c \mathcal{N}(x_i; \mu_c, \Sigma_c)$$

Since cluster memberships are unknown, we treat them as latent variables. The algorithm alternates between:
1. Estimating cluster assignments based on current parameters 2. Re-estimating parameters based on current assignments

Given current parameters θ, we compute cluster posterior probabilities (responsibilities):

$$\gamma_{c,i} = P(C_i = c|X_i = x_i) = \frac{w_c \mathcal{N}(x_i; \mu_c, \Sigma_c)}{\sum_{c'} w_{c'} \mathcal{N}(x_i; \mu_{c'}, \Sigma_{c'})}$$

For hard assignments, we assign each point to the cluster with highest responsibility:

$$c_i^* = \arg\max_c P(C_i = c|X_i = x_i)$$

Given these assignments, we can estimate new parameters by treating cluster assignments as known class labels. The log-likelihood becomes:

$$\ell(\theta) = \sum_{i=1}^{n} \log \mathcal{N}(x_i|\mu_{c_i^*}, \Sigma_{c_i^*}) + \sum_{i=1}^{n} \log w_{c_i^*}$$

This separates into two terms depending on different parameter subsets. The optimal parameters are: - Component means and covariances: empirical estimates within each cluster - Component weights: $w_c^* = \frac{N_c}{\sum_{c=1}^{K} N_c}$

When covariances are fixed to $\Sigma_c = I$ and weights to $w_c = \frac{1}{K}$, the cluster assignment rule becomes:

$$c_i^* = \arg\min_c \|x_i - \mu_c\|^2$$

This is equivalent to the K-means clustering rule, showing that GMM generalizes K-means.

The main limitation of hard assignments is that they ignore uncertainty when cluster posterior probabilities are similar across multiple components.

Soft Cluster Assignments

To address the limitations of hard assignments, we extend the algorithm to handle soft assignments where points contribute to multiple components according to their responsibilities.

Consider the log-likelihood for the observed data:

$$\sum_{i=1}^{n} \log f_{X_i}(x_i|\theta) = \sum_{i=1}^{n} \log \left(\sum_{c=1}^{K} w_c \mathcal{N}(x_i|\mu_c, \Sigma_c) \right)$$

Taking the gradient with respect to μ_c and setting to zero yields:

$$\mu_c = \frac{\sum_i \gamma_{c,i} x_i}{\sum_i \gamma_{c,i}} = \frac{F_c}{N_c}$$

where $N_c = \sum_i \gamma_{c,i}$ is the zero-order statistic and $F_c = \sum_i \gamma_{c,i} x_i$ is the first-order statistic.

This represents a weighted empirical mean where the weight of each sample corresponds to its responsibility. Similarly, for the covariance matrix:

$$\Sigma_c = \frac{1}{N_c} \sum_i \gamma_{c,i} x_i x_i^T - \mu_c \mu_c^T = \frac{S_c}{N_c} - \mu_c \mu_c^T$$

where $S_c = \sum_i \gamma_{c,i} x_i x_i^T$ is the second-order statistic.

The weights are re-estimated as:

$$w_c = \frac{N_c}{N} = \frac{\sum_i \gamma_{c,i}}{\sum_{c=1}^{K} \sum_i \gamma_{c,i}}$$

Since responsibilities depend on parameters and vice versa, we use an iterative procedure:
E-step: Estimate responsibilities given current parameters
M-step: Re-estimate parameters given current responsibilities

This procedure is a particular instance of the Expectation-Maximization (EM) algorithm.

Model Considerations and Classification

Similar to MVG, we can constrain GMM covariance matrices to be diagonal or tied across components to reduce parameters and computational costs. Proper initialization is critical for GMM performance. Common approaches include:

- K-means initialization for initial cluster assignments
- LBG algorithm: iteratively split components starting from a single Gaussian

For classification tasks, we fit one GMM per class with K_c components for class c. Class posterior probabilities are computed as:

$$P(C_t = c | X_t = x_t) = \frac{P(C_t = c) \sum_{k=1}^{K_c} w_{c,k} \mathcal{N}(x_t | \mu_{c,k}, \Sigma_{c,k})}{\sum_{c'} P(C_t = c') \sum_{k=1}^{K_{c'}} w_{c',k} \mathcal{N}(x_t | \mu_{c',k}, \Sigma_{c',k})}$$

For binary tasks, we can use the log-likelihood ratio:

$$\text{llr}(x_t) = \log \frac{\sum_{k=1}^{K_1} w_{1,k} \mathcal{N}(x_t | \mu_{1,k}, \Sigma_{1,k})}{\sum_{k=1}^{K_0} w_{0,k} \mathcal{N}(x_t | \mu_{0,k}, \Sigma_{0,k})}$$

GMMs are particularly useful for open-set multiclass classification, where the "none-of-others" class can be modeled as a GMM using unlabeled samples, providing flexibility in handling heterogeneous reject classes.